⚙️ SESSION 03

Production Model Training

Building robust, modular, and reproducible machine learning training pipelines using standard scripting architectures.

Day 1 | 11:00 AM – 12:00 PM | 60 Minutes

Session Goals & Objectives

Core Purpose

Demonstrate a compact, production-style ML training pipeline implemented as small, single-responsibility, testable Python scripts.

Emphasizes code architecture, config separation, logging, and artifact management over model complexity.

Key Learning Outcomes

By the end of this 60-minute session, learners will be able to:

- ✓ Explain reproducible training workflows
- ✓ Map script components to production duties
- ✓ Execute CLI pipelines & inspect outputs

Pipeline Folder Structure

Modular Scripts

`scripts/main.py`
Orchestration CLI

`scripts/train.py`
Model fitting & save

`scripts/evaluate.py`
Metrics computation

Data & Config

`scripts/data_loader.py`
Ingest & train/test split

`scripts/features.py`
Feature engineering

`scripts/config.yaml`
Run parameters

Interactive Alternative

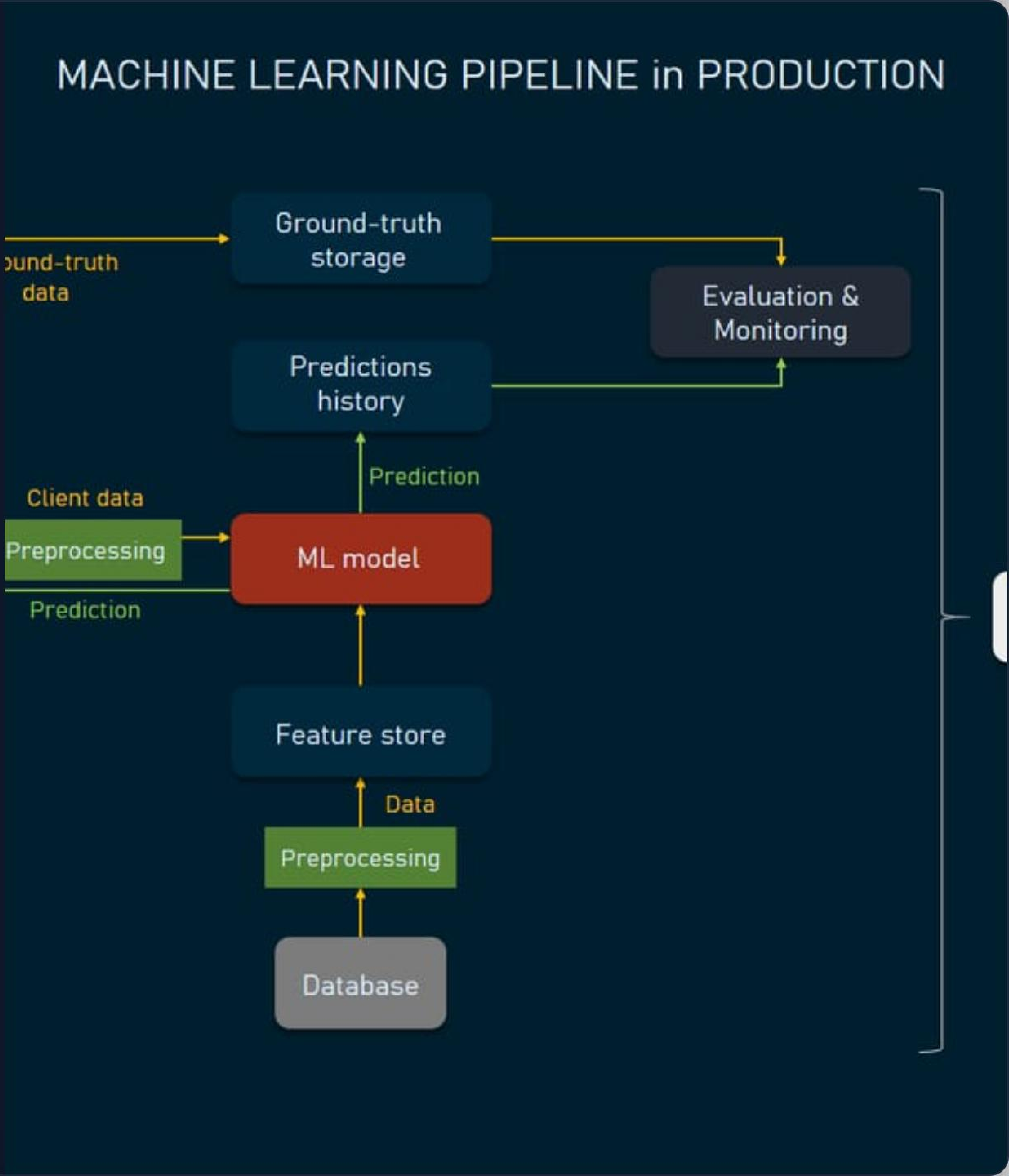
`notebook.ipynb`
Interactive notebook version used to teach the exact same architectural pattern step-by-step.

Architecture Overview

A production training pipeline isolates concerns into dedicated modules rather than monolithic scripts.

Key Design Principle

Decouple configuration from logic so experiments can be executed and audited without code changes.



Environment & Setup

Recommended Virtual Env

Create and activate virtual environment:

```
# Create venv python -m venv .venv # Activate (Windows)
.\\.venv\\Scripts\\activate # Activate (macOS/Linux)
source .venv/bin/activate pip install -r
requirements.txt
```

Lightweight Minimal Install

For quick standalone testing of this folder:

```
pip install pandas scikit-learn joblib pyyaml
```

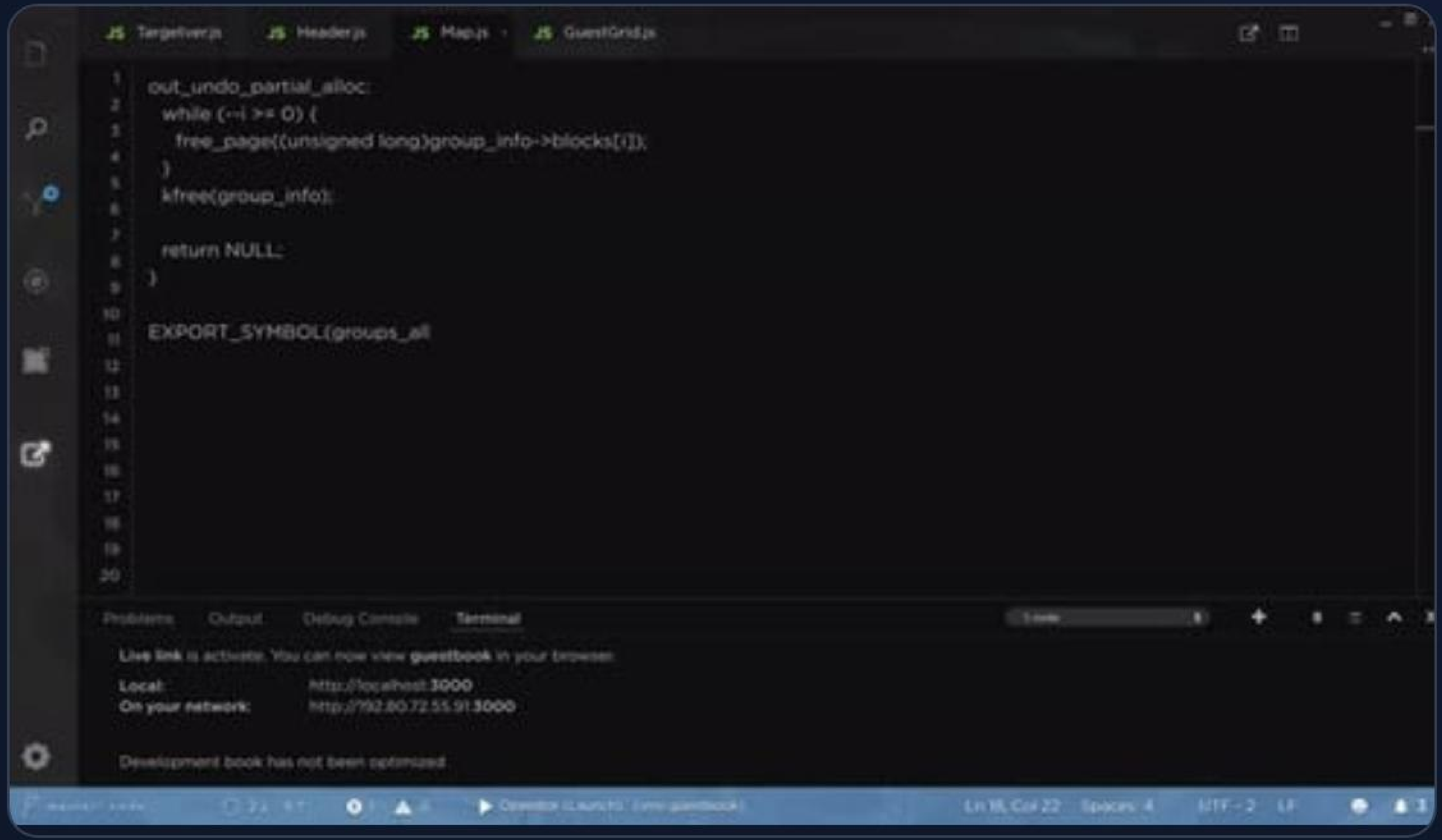
Installs minimal runtime dependencies required for dataset creation, model fitting, and YAML config parsing.

Pipeline Execution Flow

Run the pipeline from the repository root using the orchestrator command:

```
python day1/03_production_training/scripts/main.py \ --config
day1/03_production_training/scripts/config.yaml
```

- **Automated Fallback:** Creates synthetic dataset if CSV at `data.path` is missing.
- **Artifact Output:** Writes outputs automatically into the `artifacts/` directory.



Generated Run Artifacts

Trained Model

`artifacts/model.joblib`

The trained classifier serialized and persisted with `joblib`, ready for downstream inference service deployment.

Metrics Report

`artifacts/metrics.json`

Evaluation metrics JSON containing accuracy, precision, recall, F1-score, and full classification matrix.

Execution Logs

`artifacts/training.log`

Structured INFO-level run log tracking execution timestamps, parameters, steps, and progress output.

Configuration System

```yaml
# scripts/config.yaml data: path: "data/churn.csv"
target_column: "churn" test_size: 0.2 random_state: 42 model:
type: "logistic_regression" max_iter: 200 paths: model_path:
"artifacts/model.joblib" report_path: "artifacts/metrics.json"
log_path: "artifacts/training.log"
```

Hyperparameters & Data

Controls target outcomes, split ratio, seed reproducibility, and classifier options.

Explicit Paths

Centralizes artifact locations so model files and metrics write predictably without hardcoding.

Experiment Drive

Allows instructors to demonstrate repeatable experiments simply by modifying YAML fields.

Component Responsibilities

Module	Production Responsibility	Key Concept Demonstrated
`data_loader.py`	Data acquisition, validation & train/test split creation	Data integrity and reproducible random seed splitting
`features.py`	Preprocessing, imputation & one-hot encoding	Separation of feature engineering from training logic
`train.py`	Model selection, fitting & persistence routines	Encapsulated model training and artifact saving
`evaluate.py`	Classification metrics calculation & report exporting	Standardized evaluation output format (JSON)
`main.py`	CLI orchestration & centralized logging setup	User-facing interface for end-to-end runs

Instructor Demo Walkthrough

1. **Inspect Config**
   Open `config.yaml`; explain data, model, and path definitions.
2. **REPL Loader**
   Run `data_loader.py` in REPL to view columns & splits.
3. **First Run**
   Execute `main.py` to produce initial `artifacts/` files.
4. **Experiment**
   Modify `test_size` or hyperparameters & re-run pipeline.
5. **Extension**
   Challenge: Add `RandomForest` option into `train.py`.

Troubleshooting Guide

ModuleNotFoundError

Ensure your virtual environment is active and required packages are installed via `pip install -r requirements.txt` or `pip install pandas scikit-learn joblib pyyaml`.

Target Column Missing

If `target_column` is not found in CSV, open the CSV specified in `config.yaml` to inspect headers and adjust the config value.

Log Inspection

Execution progress and detailed error traces are logged directly to `paths.log_path` — tail that file to follow run progress in real-time.

SESSION SUMMARY & NEXT STEPS

Ready for Production Model Training

You now have a clean, configuration-driven, and testable machine learning training pipeline ready to run and customize.

Execution Wrappers

Use `run_demo.bat` or `run_demo.sh` for 1-click execution.

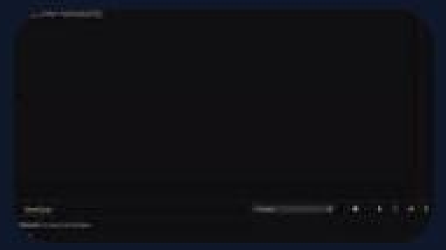
Instructor Cheat-Sheet

Refer to `explain.md` for key talking points and slide notes.

Image Sources

https://www.altexsoft.com/static/blog-post/2023/11/a0915a8c-94e5-4798-9980-46cb3751e078.jpg

Source: www.altexsoft.com



https://elements-resized.envatousercontent.com/elements-video-cover-images/files/151d1769-8dd0-4300-8252-33b671f2c307/inline_image_preview.jpg?w=500&cf_fit=cover&q=85&format=auto&s=0114fe056d2c0dea864cf78a8996099abacdbf2d8000de0c0ad72b93ad0567cf

Source: elements.envato.com